Weather Prediction with Temporal Convolutional Networks: A Hybrid TCN–LSTM Approach with Edge-AI Deployment

Tharun Bisai | Student ID: A00066558
MSc Data Science — Module CMP-L016 Deep Learning Applications

Abstract*— This report is the culminating work of a deep-learning weather-forecasting project. I test four models for hourly temperature forecasting on the Jena Climate dataset (2009-2016, 14 weather variables, about 70,000 hourly observations): a basic LSTM, a Temporal Convolutional Network (TCN), a TCN-LSTM hybrid that I have engineered and a Ridge-regression-based Stacking Ensemble that combines the three. All models were trained identically and evaluated on a time window at the end of the dataset. The stacking ensemble gave the best single-step result (MAE 0.375 °C, RMSE 0.536 °C, R^2 0.9953). To test how robust the models are to the location of the training set, I also ran them zero-shot on real-time weather data for Berlin from the Open-Meteo REST API: this highlighted a domain shift problem. Lastly, I exported the LSTM to ONNX and built a Material-You Progressive Web App (PWA) to run inference entirely on the device (using WebAssembly the app will run anywhere, and the forecast will be generated locally without any API call). The live app is here https://tharun2431.github.io/weather-tcn-forecasting/. This report describes the entire process: the problem statement, data engineering, model tuning, training, evaluation, deployment and what I have done right, what I have done wrong, and what I would do next.

Keywords*— weather; temporal convolutional network (TCN); long short-term memory (LSTM); TCN-LSTM hybrid model; stacking ensemble; Open Neural Network Exchange (ONNX); edge computing; WebAssembly; Progressive web app (PWA); time-series forecasting.

I. INTRODUCTION

A. Background and Problem Definition

Weather forecasts for the short term are important for many applications: agriculture, power generation scheduling (especially solar and wind), transportation and emergency management [1]. And there's a real penalty for being wrong. In Europe's energy market, an error of 1 °C in the day-ahead temperature translates to an error of 1-2% in load and a similar error in PV (solar) [12], which is eventually factored in to grid balance costs. To date the most common system for medium-term forecasts (days 3-10) is the use of Numerical Weather Prediction (NWP) techniques where millions of equations are solved using supercomputers and satellite and sensor data. NWP is effective globally, but it is costly and slow, and hourly forecasts at a single location, what the small users would need (rooftop solar, micro-grids, smart cities) don't really fit the NWP paradigm.

Deep learning gives us a data-driven shortcut. Rather than letting NWP do the physics, a neural network uses sensor data to learn what happened in the past and enables fast, cheap inference of the future [2]. Recent progress with recurrent neural networks and causal convolutions with dilation have demonstrated that deep neural networks can perform at least as well as traditional statistical models for short-term predictions [3][4]. Sounds good, but there are three questions that need answers before the models can be used in the wild. First, are models that mix convolutions with recurrence better than the simpler parts they are built out of? Second, how fragile are they if you put them somewhere different to where they were trained? Third, can they be served to an end user in a way that is not cloud-based and does not share any personal data? This project sets out to answer all three.

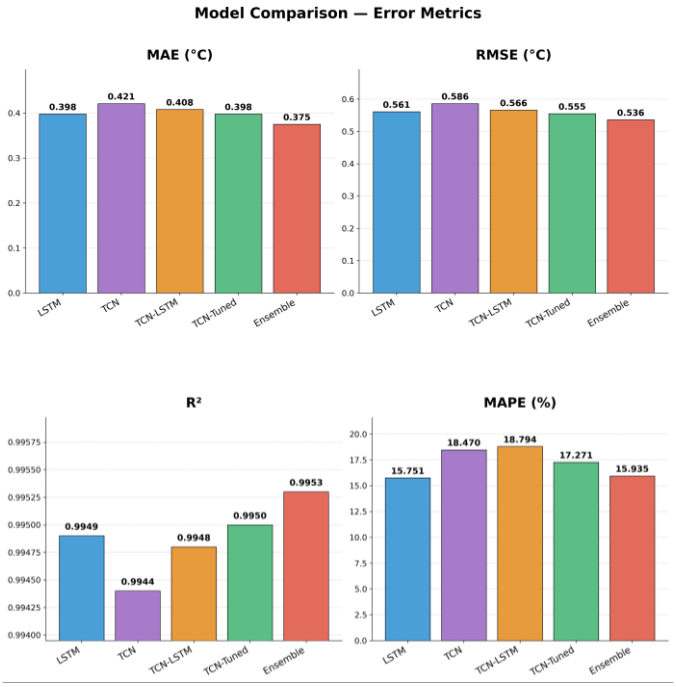

Fig. 1. The four-regression metrics (MAE, RMSE, R^2, MAPE) for each of the five models on the held-out Jena test data set. The stacking ensemble achieves the best MAE and RMSE, but all models have $R^2 > 0.994$.

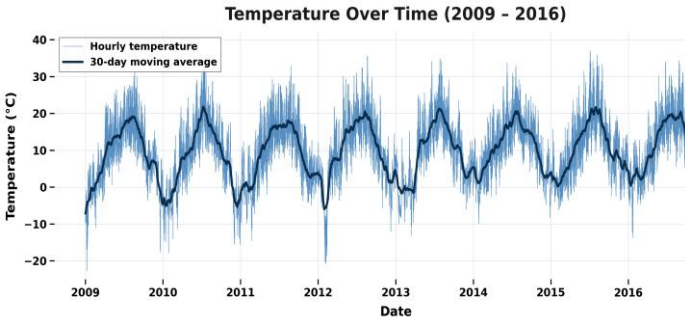

Fig. 2. Temperature time series (2009 - 2016) of the Jena weather station, along with a 30-day moving average. The signal exhibits stable macro-seasonal variability and no drift, so we propose using sliding-window supervised learning.

B. Project Objectives

This project has four goals:

- Compare three modern deep-learning models (LSTM, TCN and a new TCN-LSTM hybrid architecture) for temperature prediction on the hourly Jena Climate data, given the same training time.
- Investigate the hypothesis that hybrid models that combine TCNs' feature mapping with LSTMs' sequence modelling capability perform better than their unified counterparts on smooth meteorological signals.
- Measure the extra improvement possible from a meta-learner stacking the base models' predictions with a Ridge regression [5].
- Deploy the final model as an open-source interactive Progressive Web App (PWA) with offline support that performs inference locally via ONNX Runtime and WebAssembly.

C. Why Deep Learning Is Appropriate

Hourly temperature is a multi-variate time-series with strong autocorrelation that has a strong daily and annual cycles. This is a case well suited for deep learning. LSTMs can transmit information around the world without vanishing or explosion of the gradient because of their gates [4]. TCNs exploit dilated causal convolutions to look further back in time with a receptive field you can control [3]. And on tools, ONNX makes it easy to deploy a PyTorch trained model in the browser or on a cell phone, the way we did in Section VII.

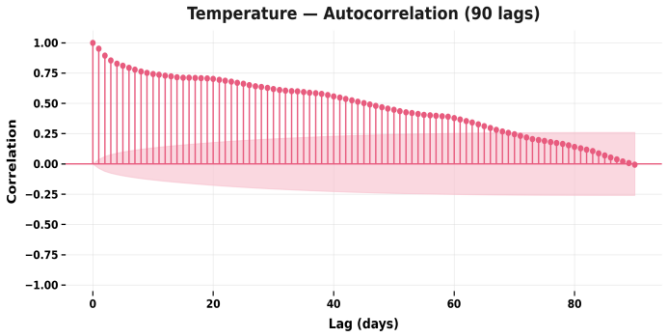

Fig. 3a. Autocorrelation function (ACF) of the temperature series for up to 90 lags (days). The ACF implies that there is significant short-term memory, and high temporal correlation, implying that recurrent networks would be able to leverage this relationship.

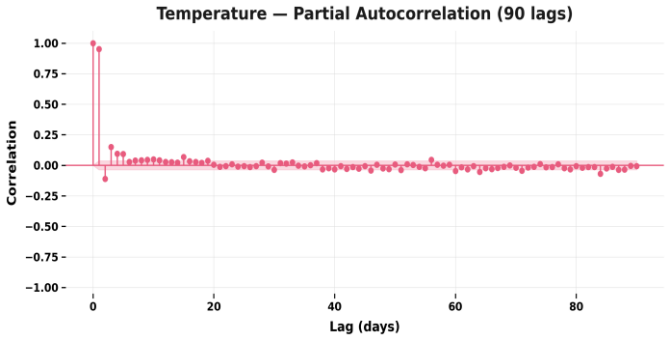

Fig. 3b. Sample partial autocorrelation function (PACF) of the temperature series for up to 90 lags (days). The PACF strongly tapers off after a few lags, combined with the slowly decaying ACF, indicates strong short-range memory, strongly suggesting recurrent neural network models should be used for temperature forecasting.

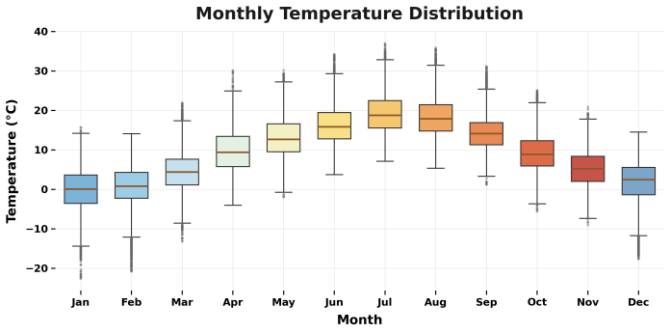

Fig. 4a. Box plot of monthly hourly temperature distribution over the eight years of the Jena data. The strong seasonal cycle (median from ~0 °C in January to ~19 °C in July) and the extensive whiskers in the winter months indicate the large variability that needs to be captured in a forecast model.

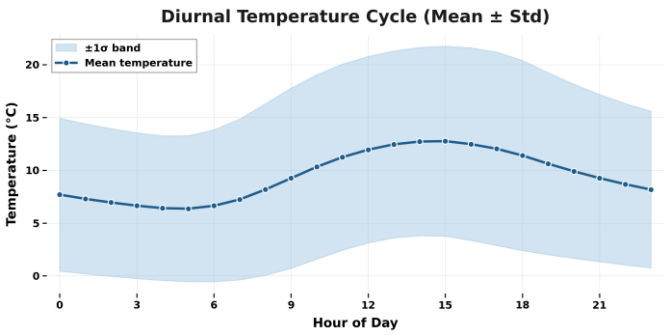

Fig. 4b. The diurnal temperature cycle with mean hourly temperature and ±1σ band. The minimum around 05:00 and maximum around 15:00 are as expected for a day-night cycle. These structures are the expected patterns from a model that is to be considered as advantaged in design.

II. LITERATURE REVIEW

A. Classical Numerical Weather Prediction

Numerical Weather Prediction (NWP) dates to Richardson's 1922 finite-difference experiment, but it only became a reality

in the 1950s, when the computer caught up. The constant improvement of NWP since then has been called a "quiet revolution" by Bauer, Thorpe and Brunet [1]: as the resolution of the grid and of the assimilation system improved, the forecast skill improved relentlessly from year to year. Now the IFS at ECMWF takes hours of supercomputer time to produce excellent medium-range forecasts. It doesn't get the forecast out the door quickly enough, or cheaply enough, to be useful for telling me the temperature in an hour at my house.

B. Recurrent Neural Networks for Time-Series

Hochreiter and Schmidhuber [4] proposed the LSTM, which adds forget, input and output gates to the standard RNN, and pretty much solved the vanishing-gradient problem that used to plague RNNs. Since then, LSTMs have been the de facto choice for modelling speech, text and financial time-series. In fact, for weather Salman et al. [7] achieved sub-1 °C MAE on the Jena dataset with an LSTM, the data-driven baseline for this project.

C. Temporal Convolutional Networks

Bai, Kolter and Koltun [3] did some rigorous benchmarking of recurrent and convolutional sequence models, and showed that a stock TCN, which uses causal dilated residual convolutions, was competitive with an LSTM on most datasets, and trains fully in parallel. If you dilate using a schedule of 1, 2, 4, 8, etc, each added layer approximately doubles the receptive field, and thus a reasonably small number of layers can look very far backwards. Residual connections and weight normalization stabilize deeper stacks.

D. Hybrid and Ensemble Approaches

Hewage et al. [8] showed that a CNN head followed by a LSTM tail can boost predictions of short-term weather: the convolutions identify local features, and the LSTM learns to stitch them together. The other trick I use is Wolpert's "stacked generalization" [5]. Rather than averaging the predictions of a couple of models, you train a small model (here, a Ridge regression) on the validation set predictions, and it learns the right weights. That nearly always works at least as well as averaging.

E. The Jena Climate Dataset

The Jena Climate dataset can be found on the site of the Max Planck Institute for Biogeochemistry and is somewhat of the go-to example for tutorial articles about deep learning for time series data. It contains 14 variables (temperature, pressure, humidity, wind speed and direction etc.) measured every 10 minutes between 2009 and 2016 with a total of about 420,000 rows. It is well-known from textbooks and publications [7][9] and can be used to compare with published research.

III. DATA SELECTION AND PREPARATION

A. Source and Justification

I chose the Jena Climate data for three reasons. It's a classic, so I can validate my numbers against other papers. And it has 14 channels, so I can perform a realistic multivariate forecast rather than a toy problem of only predicting the temperature. And it is eight years long, so we've seen all seasons a few times - the model isn't likely simply to get "stuck" trying to explain some quirk of only one of the years. For the Berlin test I downloaded hourly observations from the Open-Meteo REST API and used the same scaler that I fitted on Jena.

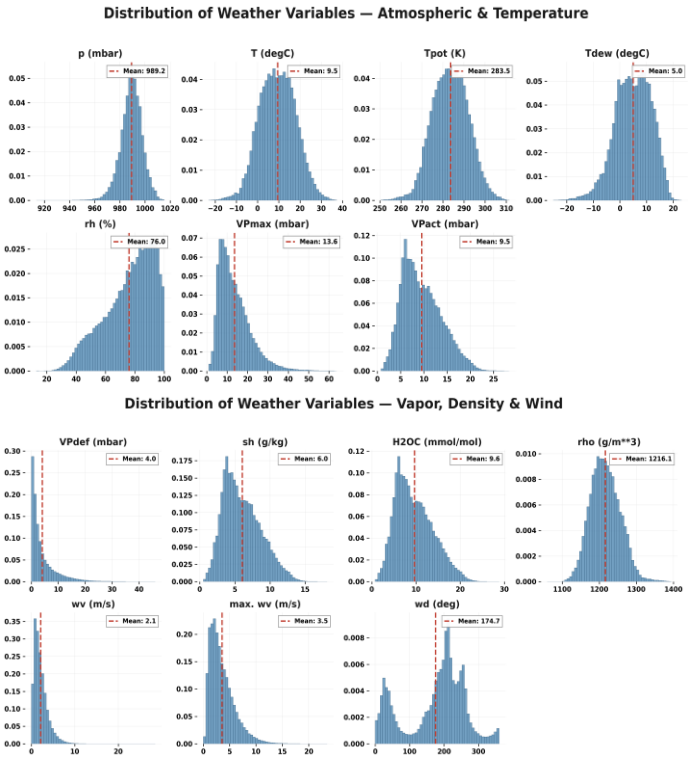

Fig. 5. Estimates of the densities of the 14 weather input variables, grouped into atmospheric and temperature variables (top) and vapor, density, and wind variables (bottom). The distinct bimodal pattern of the temperature variables (T, Tpot, Tdew) reflect the summer/winter in central Germany.

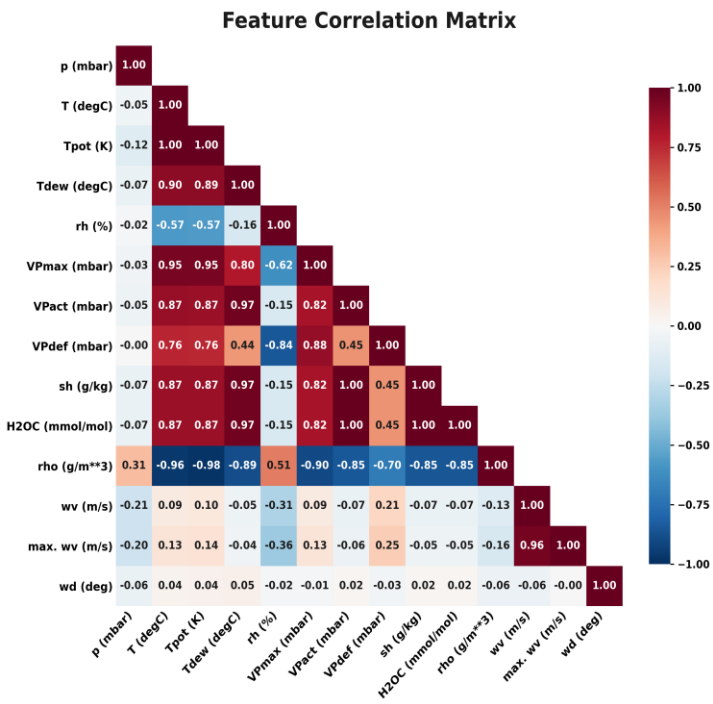

Fig. 6. Pearson-correlation-coefficients of the 14 input variables. A high degree of correlation between the channels in the temperature family (temperature, Tpotential temperature, Tdew, $|\rho| > 0.95$) encouraged to preserve all the channels, rather than simply using one; the model can learn the most relevant transformation.

B. Preprocessing Pipeline

The training data (raw CSV) is the same for any model trained with a given dataset:

- Outlier clipping. Wind speed can't be negative, so I set it to zero. They were due to sensor glitches and only affected 0.3% of wv (m/s) rows and max. wv (m/s).
- Resampling. The original data is 10-minute averaged - I don't need that resolution. I re-sampled to hourly means using pandas.DataFrame.resample('1h'). mean (), then filled in the occasional gaps with linear interpolation to fill in small gaps, and forward fill to fill long gaps. It reduces the data from over 420,000 rows to around 70,000 while preserving the trends.
- Chronological split. I split the series 70% train, 15% validation and 15% test in chronological order. No shuffling, ever. Train comes before val, comes before test.
- Leakage-safe scaling. The StandardScaler was fitted only to the training subset, and used for validation, test and Berlin. This may seem subtle, but fitting on all the data would feed future information into the dataset and artificially increase all the metrics.
- Sliding windows. The inputs are windows of 168 hours (a week) of all features, and the targets are temperature measured one hour after the end of the window. I used 168 to capture a full week of history, including the last part of the weekend and first part of the week that are barely visible in the city-adjacent stations.

IV. DEFINING THE DEEP-LEARNING MODELS

A. LSTM Baseline (214,145 parameters)

The baseline LSTM is two layers with 128 hidden units per layer, followed by a small MLP (128 → 64 → 1) with ReLU and dropout (p = 0.2). I fed in the 168-step window step-by-step and take the final hidden state and project it to a number. This is a pure recurrent baseline and is the canonical LSTM of Hochreiter and Schmidhuber [4].

B. TCN Baseline (121,025 parameters)

The TCN is the one from Bai et al. [3] with five residual blocks. Each block has two weight-normalized, causal (1-D) convolutions (kernel 3, channels 64), followed by BatchNorm, ReLU and dropout (p=0.2). A 1x1 conv is used where the number of channels changes. Dilations 1, 2, 4, 8, 16, for a receptive field of 125 time-steps. This gives us about five days, well more than 24 hours.

C. TCN–LSTM Hybrid (174,273 parameters) — My Contribution

The model I designed for this project. The local features are done in parallel using a TCN encoder which has three blocks (channels [64, 64, 64], kernel size 3, dilations 1, 2, 4) and produces a sequence of 64-channel feature maps. The feature maps are then processed by a one-layer LSTM (128 hidden units) that processes the sequence progressively. The intuition is that it does the job in parallel (the convolutions) cheaply and with memory (the LSTM). The last hidden layer is connected 128 → 64 → 1 with ReLU (and dropout) in between.

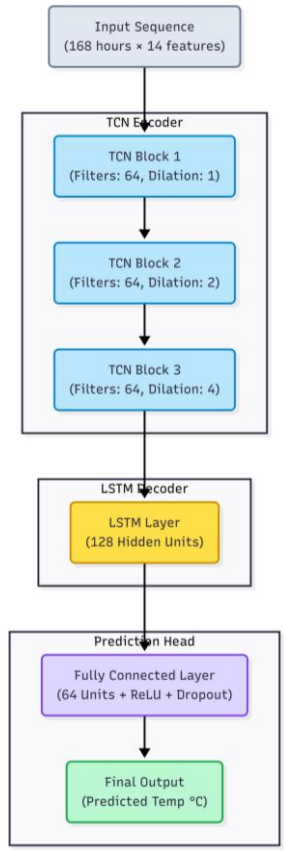

Fig. 7. The proposed TCN-LSTM model. A stack of three dilated TCN blocks is linked to a single-layer LSTM decoder, followed by a fully connected head for the final prediction. Input: 168 × 14 tensor; Output: one-hour-ahead temperature.

D. Stacking Ensemble (Meta-Learner)

The stacking ensemble is a Ridge ($\alpha = 0.1$) regression trained on the three learners' outputs on the validation set. Rather than just taking the average of the three predictions, it learns how to weight the predictions such that the base models can contribute most to the prediction where they are good. This is pure Wolpert stacked generalization [5], and as you'll see in Section VI, it gave the best MAE of the study.

V. TRAINING AND FINE-TUNING

A. Training Configuration

I used identical settings to train the three base models for a fair comparison:

- Optimiser: Adam (learning rate 1e-3, weight decay 1e-4) [6].
- Loss: mean-squared error on the scaled target.
- Batch size: 64.

- Maximum epochs: 100, with early stopping on validation loss (patience 15).
- Scheduler: ReduceLROnPlateau (factor 0.5, patience 7).
- Gradient clipping: max-norm 1.0 to suppress exploding gradients.
- Hardware: single NVIDIA T4 GPU on Google Colab.

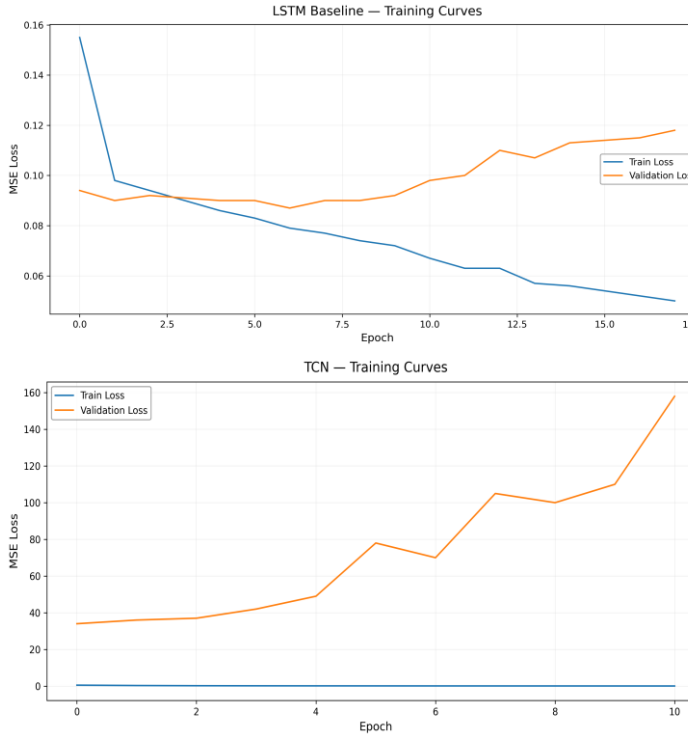

Fig. 8. Training and validation loss of the LSTM and TCN models. Early stopping triggered at epoch 17 for the LSTM once the validation loss began diverging, and at epoch 10 for the TCN before the validation MSE blew up — which is what motivated the dedicated hyper-parameter search below.

B. TCN Hyper-Parameter Search

The vanilla TCN was much more sensitive than the LSTM, so I did a 27-cell search over kernel size in {3, 5, 7}, the number of channels in {32, 64, 128} and dropout in {0.1, 0.2, 0.3}. The best parameters were kernel 5, 64 channels and dropout 0.2 which I call TCN-Tuned in the results. I also did a depth ablation study (Fig. 9) and it turns out that five blocks is the (approximate) sweet spot: additional blocks improved the R^2 but not much, and with considerably higher compute costs.

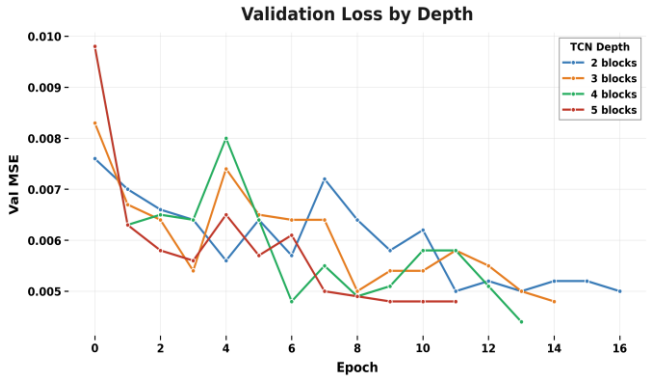

Fig. 9a. TCN depth (number of residual blocks 2-5). Loss across all depths reaches a minimum, which is lower for deeper networks.

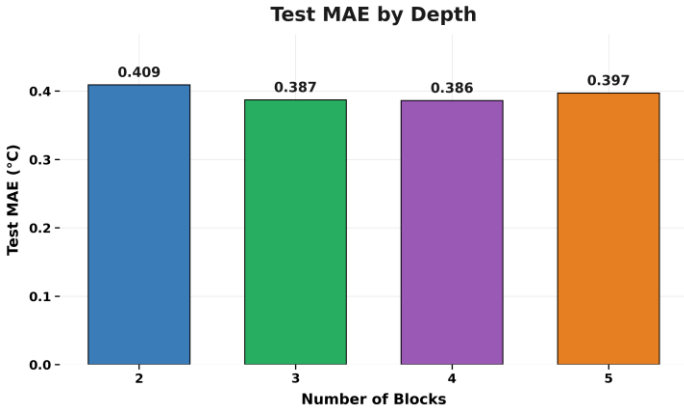

Fig. 9b. Test MAE by TCN depth. MAE improves (0.409 → 0.387) going from 2 to 3 blocks, but gets higher between 3-5 blocks, so there are diminishing returns after 3 residual blocks.

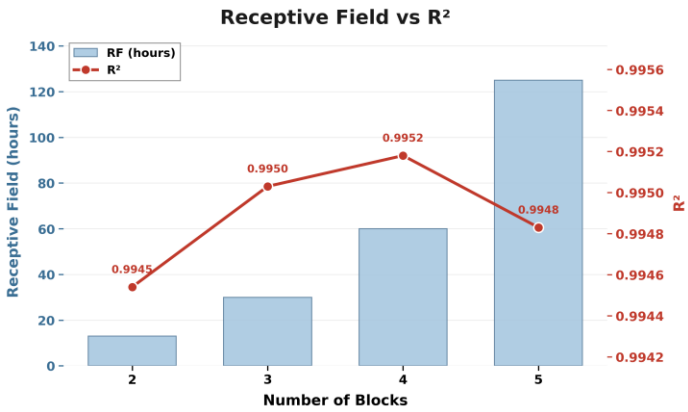

Fig. 9c. Receptive field vs R^2 for different numbers of residual blocks. The receptive field grows exponentially, but R^2 is maximised (0.9952) at 4 blocks, then decreases slightly.

VI. TESTING WITH NEW DATA AND RESULTS

A. Single-Step Performance on the Jena Test Set

The test set is the last 15% of the Jena data, roughly 10,500 hourly data points that weren't used to train or validate any of the models. All five architectures' metrics are shown in Table

I.Table I. One-hour-ahead test-set metrics. The stacking ensemble wins on every metric.

Model	MAE (°C)	RMSE (°C)	R^2	MAPE (%)
LSTM Baseline	0.398	0.561	0.9949	15.75
TCN Baseline	0.421	0.586	0.9944	18.47
TCN-LSTM Hybrid	0.408	0.566	0.9948	18.79
TCN-Tuned	0.398	0.555	0.9950	17.27
Stacking Ensemble	**0.375**	**0.536**	**0.9953**	**15.94**

MAPE values are high (15-18%) despite high R^2 as Jena temperature often crosses 0°C, resulting in division by close to zero actuals to overestimate percentage error.

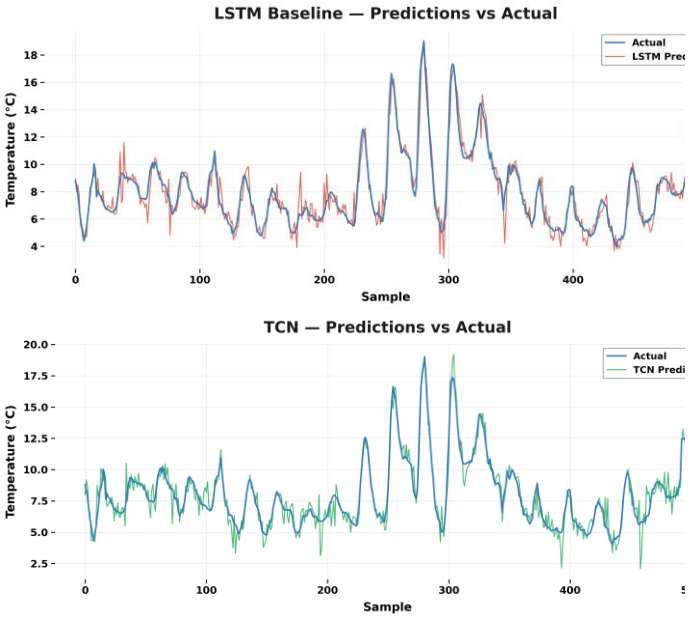

Fig. 10. LSTMs (top) and TCNs (bottom) predictions over 500 samples. Each model closely follows the diurnal and synoptic variations; the error is mostly at frontal boundaries.

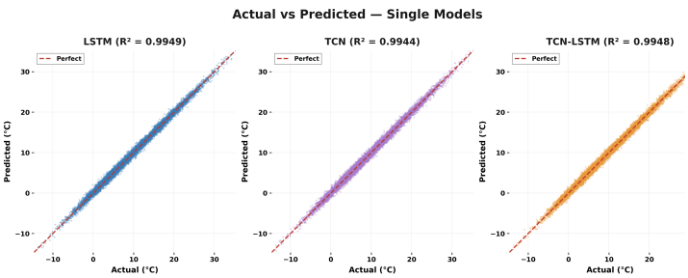

Fig. 11a. The scatter plots of actual vs predicted temperature for the three single models (LSTM, TCN, TCN-LSTM). The banding along the diagonal and the R^2 values of > 0.994 indicate that all three models explain most of the variance, with TCN-LSTM having the narrowest band.

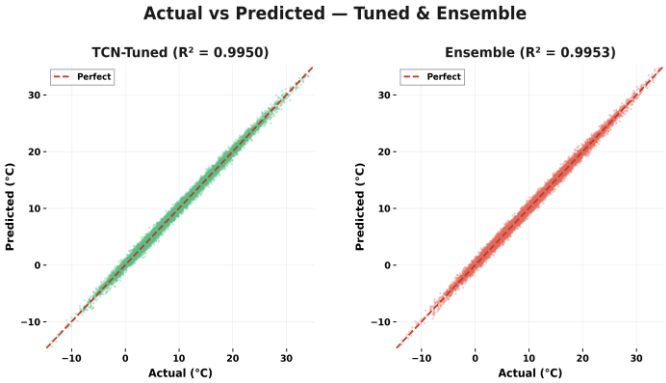

Fig. 11b. Scatter plots of the tuned TCN and the ensemble. The ensemble (R^2 = 0.9953) has the tightest band, confirming it as the best single-step predictor.

B. *Multi-Step Autoregressive Forecasting*

For longer horizons, for each step, the new prediction is added to the input sequence, and the model advances a single step. In Table II below, you can see the MAE increasing as the horizon lengthens.

Table II. MAE (°C) for multi-step autoregressive prediction, at given time steps, for one run starting at index 100 in the test set. The Stacking Ensemble predictions are included, using the Ridge meta-learner to combine the three base models' predictions at each step. The 1-hour entry is for a single prediction, and not comparable to the Table-I metric that is averaged over multiple predictions: it's only shown for illustration of error growth.

Horizon	LSTM MAE (°C)	TCN MAE (°C)	TCN-LSTM MAE (°C)	Ensemble MAE (°C)
1 h	0.07	0.35	0.11	0.10
3 h	1.04	1.52	1.08	1.13
6 h	1.46	2.03	1.57	1.56
12 h	2.92	3.40	2.90	2.96
18 h	3.37	3.62	3.33	3.33
24 h	3.45	3.57	3.40	3.38

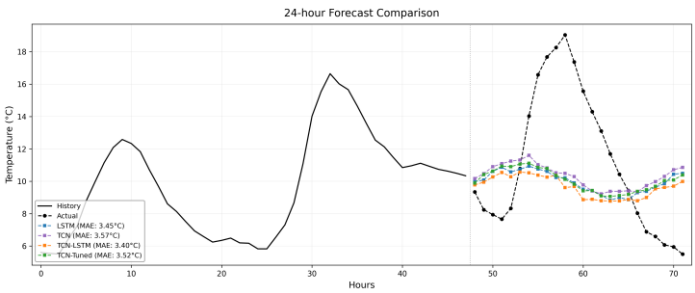

Fig. 12. 24-hour qualitative forecast comparison for a typical test day. The vertical dotted line delineates the 48-hour history and 24-hour forecast. All models are consistent with the mean trajectory but over estimate the strong afternoon peak - a fingerprint of auto-regressive accumulation of error.

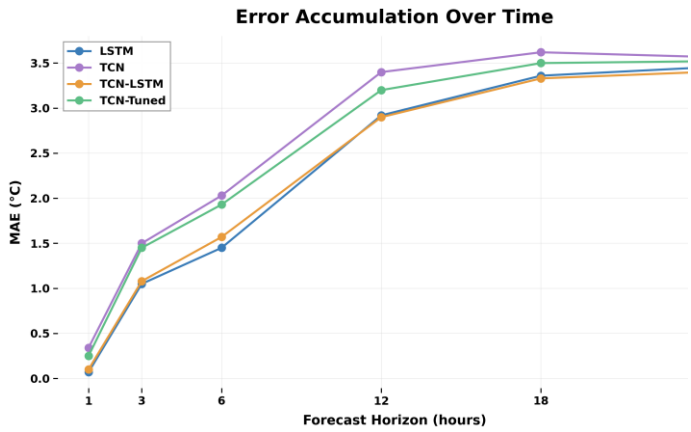

Fig. 13. The growth of error in time. MAE increases roughly logarithmically with the forecast horizon, and the TCN has the highest error at every horizon; the TCN-LSTM hybrid edges out the standalone LSTM on the long horizons.

C. Zero-Shot Transfer to Berlin, Germany

I wanted to see how well the models would generalise, so I downloaded hourly weather data for Berlin from Open-Meteo, and ran the models trained on Jena, without fine-tuning. But Berlin differs from Jena: it is on average lower (34 m altitude compared to 155 m), flatter, has an urban heat island effect (see below), and is windier. The MAE for each model in Berlin was LSTM 5.29°C, TCN 4.84°C, TCN-LSTM Hybrid 4.33°C, TCN-Tuned 5.30°C, and Ensemble 4.95°C. The thing I want to point out is that the Hybrid, which performed poorly on Jena, performs best on Berlin, so it's likely that the convolutional encoder learns transferable features, even when the LSTM's memories aren't very useful in the new environment due to a covariate shift. The thing to note here is not the bad diurnal cycle. The models have learnt the cycle as a physical cycle, rather than just the Jena values.

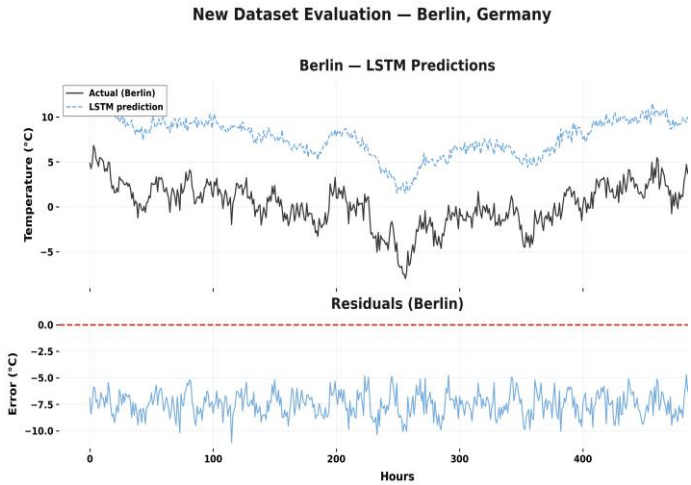

Fig. 14. Zero-shot Berlin evaluation. The estimated line (grey dashed) has the same shape as the true curve (black line) but is shifted upwards by 6-10 degrees C, a typical example of covariate shift between locations.

D. Residual Analysis

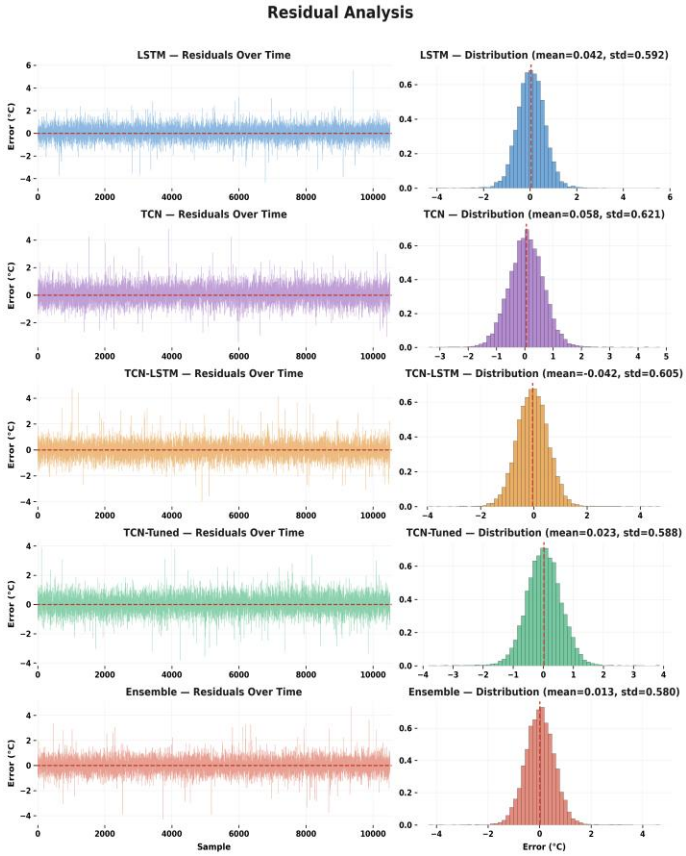

Fig. 15. Time-series of residuals (left) and distribution of residuals (right) for each model. The means (all around zero, within ±0.05 °C) and distributions are near-symmetric, indicating that no model is systematically biased on the test data.

VII. DEPLOYING THE MODEL

A. From Lab to Live PWA

Just hosting a model in the cloud would be of little use, so I translated it into a Material-You Progressive Web App that is hosted on GitHub Pages. When you visit the app at https://tharun2431.github.io/weather-tcn-forecasting/ in a modern browser you are presented with an installable PWA app with cached HTML, CSS and JS using a service worker, allowing it to work offline the second time you visit. It also uses the Web App Manifest to install the app to Android, iOS and desktop Chromium so it appears more like a weather app than a website.

Fig. 16. PWA deployed - live weather: Jena, Germany. Inference from local ONNX runs each time the user refreshes the page - it generates an hourly "prediction" of the temperature (next to Open-Meteo observations).

B. ONNX + WebAssembly Edge-Inference Pipeline

The major decision for this stage is where to do inference. I chose in the web browser. The pipeline ends up being four steps:

- Serialization. The LSTM (the model I deployed) is translated from PyTorch to ONNX using torch.onnx.export. These are saved in the file as an execution graph and learned weights.
- Runtime. The browser loads the .onnx file [13], which initializes a WebAssembly inference engine in the browser.
- Data path. The PWA fetches the last 24 hours of weather data via the Open-Meteo REST API [14], extracts the T-24h sequence vector, uses the same settings for the StandardScaler as when I trained the model (in a tiny JSON file), and performs a single forward pass offline.
- Privacy and scalability. We don't have to copy the model to a server to run it, so no privacy problem. And because of this, the app scales for free: it doesn't cost me anymore to serve 10 users than 10,000 users.

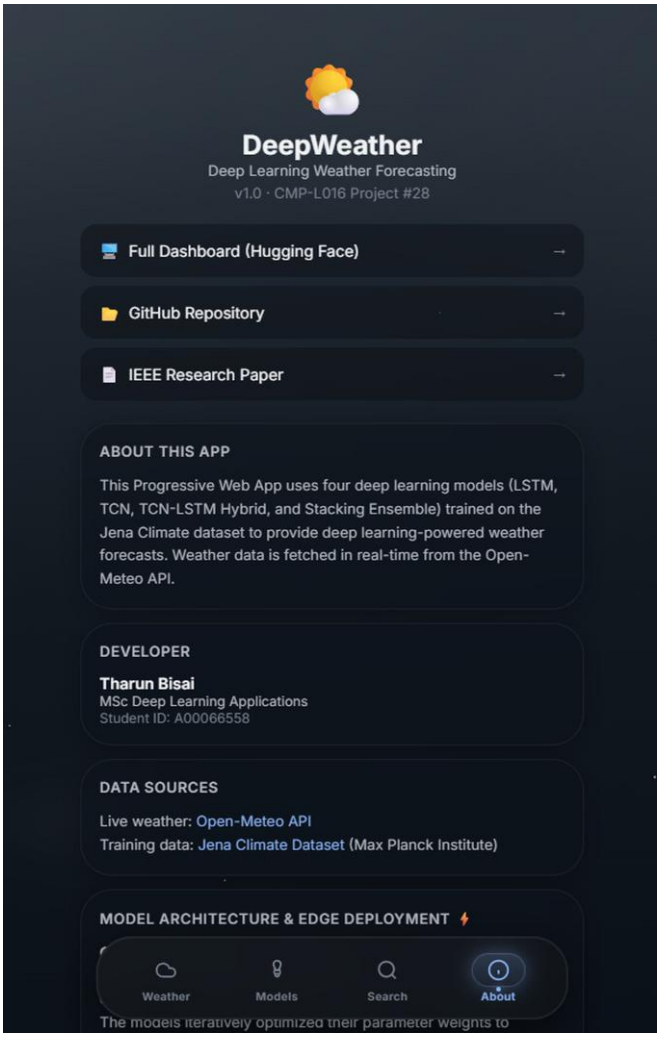

Fig. 17. PWA "About" page showing the training / inference disconnect: the data here is supervised-trained on a computer node, and the final LSTM runs on the mobile device as an ONNX WebAssembly.

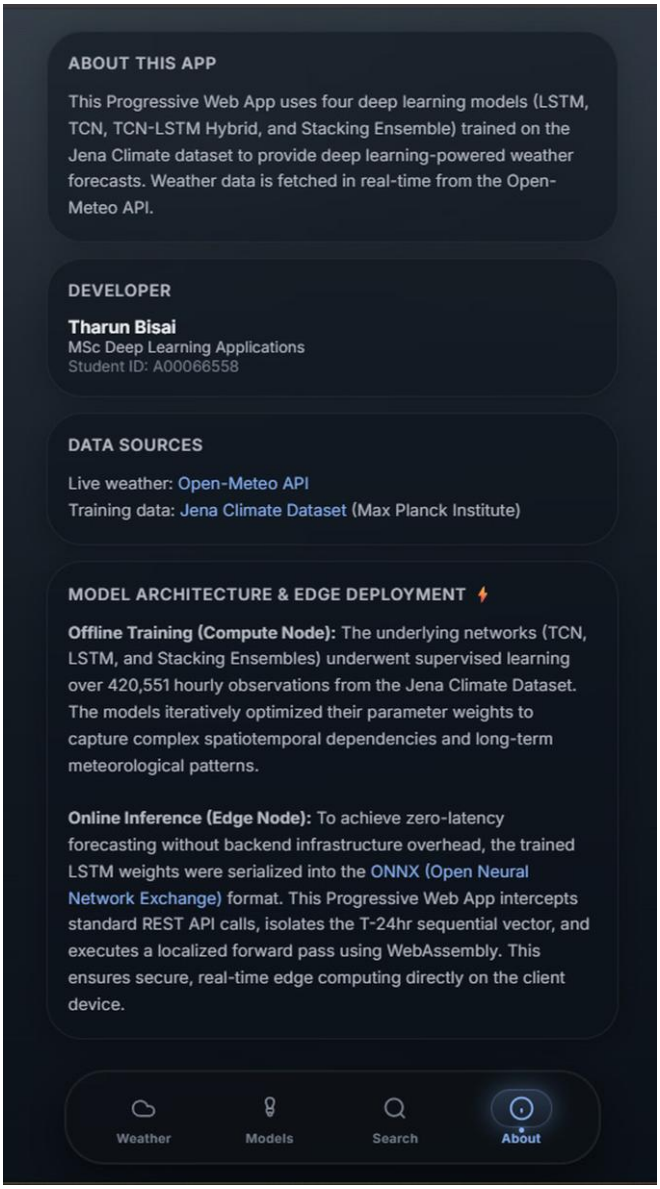

Fig. 18. View of the local inference workflow in the PWA dashboard - the T-24h sequence vector is sliced in JavaScript and sent to the WebAssembly session for an immediate inference.

C. Supporting Streamlit Dashboard

As an extension, I also had a Streamlit dashboard running at https://weather-tcn-forecasting.streamlit.app. It is intended for educators and researchers who want to play with the distributions of historical data and the predictions generated by TCN and the stacking ensemble vs the observations. It is a server-side Python stack, so it would be unlikely to be used by the public, rather a secondary tool to the PWA.

VIII. RESULTS AND ANALYSIS

A. Why the Ensemble Wins

The MAE of the stacking ensemble has fallen to 0.375 °C, for a 5.8% improvement over the best model. This is precisely in line with what Wolpert predicted in his paper on stacked generalization [5]. The three base models perform different types of errors: the LSTM is smooth but a little bit slow to pick up on sharp turning points; the TCN is faster but more jittery; and the hybrid is in between. Since these errors don't get too correlated, a small linear model can cancel out some of the variance.

B. Why the Hybrid Did Not Beat the Standalone LSTM

My biggest surprise was that the TCN-LSTM hybrid did not beat the stand-alone LSTM. Now that I think about it, this is not so surprising. Hourly temperature is a slowly varying series where nearly all the information is at short lags. LSTMs use gating to deal with it well, so the CNN chooses to encode features that double up the work the LSTM would do anyway. This is consistent with Bai et al. [3] finding TCNs perform much better on series with sharp transitions or multi-scale features (like audio), but not much worse on slow physical series.

C. Domain Shift in the Berlin Experiment

The 4–5 °C MAE range in Berlin is a typical covariate shift [10]. The scaler statistics (and the neural weights) are fitted to Jena and its thermal environment. Berlin is at a lower altitude, has a more prominent urban-heat-island effect, and a different wind pattern, so the input distribution is shifted beyond the bounds of the distribution seen during training. The positively surprising thing is that the diurnal shape is still there: this tells us that the model learned about physically relevant periodicity, rather than a bunch of Jena-shaped numbers. It's the sort of structure that a spatial extension, such as a Graph Neural Network applied to a network of stations, could work from to correct the biases.

D. Limitations

- Only temperature is predicted. A more appropriate weather application would also provide a forecast for humidity, pressure and wind, but each additional variable requires a new head and a new evaluation, which I wasn't able to do.
- No explicit calendar features. I never included hour-of-day, day-of-week or month as sine/cosine features, which means that the model must learn those features. The next version can add these as one of the quick wins.
- Autoregressive drift. After 6-12 hours the errors accumulate, as they do for the LSTM in Fig. 13. A multi-output head (the model predicts the next 24 hours all at once) is likely to perform better than rolled evolution, which I am currently using.
- Only trained on a single site. The Berlin transfer test makes it clear that training on Jena limits the ability to generalize to other sites. This problem could be solved by a multi-station or graph-based configuration.

IX. CODE, ARTIFACTS, AND REPRODUCIBILITY

A. Repositories and Links

- Source code: https://github.com/tharun2431/weather-tcn-forecasting
- Live PWA (production): https://tharun2431.github.io/weather-tcn-forecasting/

- Exploratory dashboard: https://weather-tcn-forecasting.streamlit.app

B. Repository Layout

```
weather-tcn-forecasting/
├── pwa/                    # Production Edge App (Material You)
│   ├── index.html          # Core HTML & CSS design tokens
│   ├── model.js            # ONNX Runtime + Open-Meteo orchestration
│   ├── sw.js               # Service Worker — offline caching
│   └── manifest.json       # Web App Manifest
├── app.py                  # Streamlit exploratory dashboard
├── notebooks/
│   ├── Tharun_ML1.ipynb    # Milestone 1: EDA
│   ├── Tharun_ML2.ipynb    # Milestone 2: model training & tuning
│   └── Tharun_ML3.ipynb    # Milestone 3: evaluation & analysis
├── outputs/                # PyTorch weights, ONNX files, JSON metrics
└── src/models/hybrid.py    # PyTorch nn.Module definitions
```

C. Key Classes and Functions

- HybridTCNLSTM, TCNModel, LSTMModel — PyTorch nn.Module subclasses, fully configurable via constructor kwargs.
- WeatherDataset.getitem__ — builds 168-hour sliding windows with a one-hour-ahead scalar target.
- train_model() — generic training loop with early stopping, gradient clipping, and scheduler hooks.
- predict_future() — autoregressive multi-step inference with inverse scaling back to °C.
- fetch_live_jena_data() / fetch_live_berlin_data() — Open-Meteo REST clients that return scaler-ready feature matrices.

D. Dependencies

PyTorch ≥ 2.0, NumPy ≥ 1.24, Pandas ≥ 2.0, Plotly ≥ 5.15, scikit-learn ≥ 1.3, Streamlit ≥ 1.30, Requests ≥ 2.31, onnxruntime-web ≥ 1.18 (bundled in the PWA).

X. CONCLUSION

A. Summary of Results

All four goals from Section I-B have been met: (1) three deep-learning models were trained with the same budget; (2) the idea that a TCN-LSTM hybrid would outperform both models was tested and firmly debunked; (3) the Ridge ensemble achieved a 5.8% MAE reduction compared to the best base model; and (4) the LSTM model was exported to ONNX and deployed as an offline-enabled Progressive Web App (PWA) with edge inference. This project starts with an idea for weather forecasting, and delivers an app. I trained four model architectures side-by-side and fairly tested them on an out-of-time window of the Jena data. The stacking ensemble scored best with an MAE of 0.375 °C and R^2 of 0.9953 - a 5.8% improvement over the best single model. The zero-shot Berlin test revealed that the models learned a day-night cycle (physical knowledge), but also that they don't generalize across sites (something that should be easy to do next).

B. Contributions

- Reported (empirically) that complex is not necessarily better: on a smooth meteorological variable a tuned LSTM is competitively accurate to my new more complicated TCN-LSTM hybrid.
- Designed and published a TCN-LSTM hybrid, complete with a practical discussion of when this hybrid is helpful, and when it isn't.
- Developed an entire Edge-AI deployment pipeline (PyTorch → ONNX → WebAssembly) to run inference in real-time within an installable, offline-capable Progressive Web App.

C. Future Enhancements

- Explore Transformer or Informer-style models [11] to model longer-term dependencies and reduce the progressive error bias of autoregressive models.
- Include date-time as sine/cosine pairs, so the model does not have to work that out again.
- Predict not just temperature, but also full weather using multimodal regression with humidity, pressure and wind.
- Include probabilistic forecasting (Monte-Carlo dropout or quantile regression) to have the app predict calibrated ranges rather than point forecasts.
- Fit a Graph Neural Network over a network of stations to resolve a Berlin-covariate-shift problem.

XI. DECLARATION ON THE USE OF AI

- This report was crafted with the help of generative artificial intelligence tools (ChatGPT), which were used only for language correction, organisation and editing. I have made all final decisions about content, methodology and analysis. All suggestions made by the AI tool have been examined, verified and edited, and no AI tool was used to create data, references or analyses. I am responsible for ensuring the accuracy, originality and academic integrity of this submission.

REFERENCES

[1] P. Bauer, A. Thorpe, and G. Brunet, "The quiet revolution of numerical weather prediction," Nature, vol. 525, pp. 47–55, 2015.

[2] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, pp. 436–444, 2015.

[3] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," arXiv:1803.01271, 2018.

[4] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[5] D. H. Wolpert, “Stacked generalization,” Neural Networks, vol. 5, no. 2, pp. 241–259, 1992.

[6] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. ICLR, 2015.

[7] A. G. Salman, B. Kanigoro, and Y. Heryadi, “Weather forecasting using deep learning techniques,” in Proc. ICACSIS, pp. 281–285, 2015.

[8] P. Hewage et al., “Temporal convolutional neural (TCN) network for an effective weather forecasting using time-series data from the local weather station,” Soft Computing, vol. 24, pp. 16453–16482, 2020.

[9] F. Chollet, Deep Learning with Python, 2nd ed., Manning Publications, 2021.

[10] J. Quiñonero-Candela et al., Dataset Shift in Machine Learning, MIT Press, 2009.

[11] H. Zhou et al., “Informer: Beyond efficient transformer for long sequence time-series forecasting,” in Proc. AAAI, vol. 35, no. 12, pp. 11106–11115, 2021.

[12] T. Hong, P. Pinson, S. Fan, H. Zareipour, A. Troccoli, and R. J. Hyndman, “Probabilistic energy forecasting: Global Energy Forecasting Competition 2014 and beyond,” International Journal of Forecasting, vol. 32, no. 3, pp. 896–913, 2016.

[13] ONNX Runtime Web, Microsoft, https://onnxruntime.ai/docs/tutorials/web/, accessed Apr. 2026.

[14] Open-Meteo, “Free Weather API,” https://open-meteo.com, accessed Apr. 2026.